

Assignment No: 4(A)

```
#include <stdio.h>
#include <cuda_runtime.h>
#include <stdlib.h>

__global__ void vectorAdd(int *A, int *B, int *C, int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if(i < n)
    {
        C[i] = A[i] + B[i];
    }
}

int main()
{
    int n = 1000;
    int size = n * sizeof(int);

    int *A, *B, *C;
    int *d_A, *d_B, *d_C;

    // allocate host memory
    A = (int*)malloc(size);
    B = (int*)malloc(size);
    C = (int*)malloc(size);

    for(int i=0;i<n;i++)
    {
        A[i] = i;
        B[i] = i*2;
    }

    // allocate device memory
    cudaMalloc((void**)&d_A, size);
    cudaMalloc((void**)&d_B, size);
    cudaMalloc((void**)&d_C, size);

    // copy data to GPU
    cudaMemcpy(d_A, A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, B, size, cudaMemcpyHostToDevice);

    // kernel launch
    vectorAdd<<<(n+255)/256, 256>>>(d_A, d_B, d_C, n);
```

```

// copy result back
cudaMemcpy(C,d_C,size,cudaMemcpyDeviceToHost);

printf("Vector Addition Result:\n");

for(int i=0;i<10;i++)
{
    printf("%d + %d = %d\n",A[i],B[i],C[i]);
}

cudaFree(d_A);
cudaFree(d_B);
cudaFree(d_C);

free(A);
free(B);
free(C);

return 0;
}

```

Output:

Running in FUNCTIONAL mode...

Compiling...

Executing...

Vector Addition Result:

0 + 0 = 0

1 + 2 = 3

2 + 4 = 6

3 + 6 = 9

4 + 8 = 12

5 + 10 = 15

6 + 12 = 18

7 + 14 = 21

8 + 16 = 24

9 + 18 = 27

Exit status: 0